

A Reconfigurable Accelerator for Smith–Waterman Algorithm

Xianyang Jiang, Xinchun Liu, Lin Xu, Peiheng Zhang, and Ninghui Sun

Abstract—Scanning bio-sequence database and finding similarities among DNA and protein sequences is basic and important work in bioinformatics field. To solve this problem, Needleman–Wunschh (NW) algorithm is a classical and precise tool, and Smith–Waterman (SW) algorithm is more practical for its capability to find similarities between subsequences. Such algorithms have computational complexity proportional to the length product of both involved sequences, hence processing time becomes insufferable due to exponential growth speed and great amount of bio-sequence database. To alleviate this serious problem, a reconfigurable accelerator for SW algorithm is presented. In the accelerator, a modified equation is proposed to improve mapping efficiency of a processing element (PE), and a special floor plan is applied to a fine-grain parallel PE array and interface components to cut down their routing delay. Basing on the two techniques, the proposed accelerator can reach at 82-MHz frequency in an Altera EP1S30 device. Experiments demonstrate the accelerator provides more than 330 speedup as compared to a standard desktop platform with a 2.8-GHz Xeon processor and 4-GB memory and has 50% improvement on the peak performance of a transferred traditional implementation without using the two special techniques. Our implementation is also about 9% faster than the fastest implementation in a most recent family of SW algorithm accelerators.

Index Terms—Bioinformatics, computational complexity, field-programmable gate array (FPGA), reconfigurable accelerator, Smith–Waterman (SW) algorithm.

I. INTRODUCTION

SCANNING bio-sequence database to find similarities among DNA and protein sequences is basic work in bioinformatics field. It allows biologists to discover sequences sharing common subsequences between a query sequence and a subject sequence, and leads to identify similar functionalities among sequences from a biological standpoint. However, the operation is a serious challenge to available computing resource. This is because that an exponential growth speed and great amount of bio-sequence banks (e.g., a genomic or protein database, etc.) lead to a much faster improvement in absolute computing-ability requirements than the expected speedup gain on a general processor predicted by Moores law.

Manuscript received August 1, 2007. This work was supported by the NSFC under Grant 60403025 and 60573163, and by PRA under Grant S104-04. This paper was recommended by Associate Editor S. Tsukiyama.

X. Jiang is with Key Laboratory of Computer System and Architecture, Institute of Computing Technology, Chinese Academy of Sciences, 100080 Beijing, China, and also with the Institute of Microelectronics and Information Technology, Wuhan University, 430072 Wuhan, China (e-mail: xyjiangict@gmail.com; xyjiang@ncic.ac.cn).

X. Liu, L. Xu, P. Zhang, and N. Sun are with Institute of Computing Technology, Chinese Academy of Sciences, 100080 Beijing, China (e-mail: lxc@ncic.ac.cn; xulin@ncic.ac.cn; zph@ncic.ac.cn; snh@ncic.ac.cn).

Digital Object Identifier 10.1109/TCSII.2007.909857

Many search algorithms aim to solve this application problem. Normally, these algorithms have quadratic computational complexity with respect to the length of involved sequences. One method to alleviate the time-consuming problem is to introduce heuristics [1]. Though heuristics are renowned methods in this domain, all heuristics have an outstanding shortcoming that their alignment quality decreases dramatically along with time-efficiency increasing a little. In addition, some search algorithms can not benefit from heuristics at all [2].

Another approach is to utilize parallel technologies while getting high quality results in reasonable time. To utilize parallelism, one has various possibilities: parallel computer, workstation, dedicated hardware, etc. These possibilities can provide equivalent numerical results, but the performance-cost ratio of the first two is often low. With a high performance-cost ratio, dedicated hardware [3]–[5] has been paid much attention to. Its high-performance roots in a proper architecture adapted to a certain algorithm. Though having this advantage, such architecture is lack of flexibility to adapt to different search algorithms.

For the search problem, Needleman–Wunschh (NW) algorithm [6] is a classical and precise tool. Compared to NW algorithm, Smith–Waterman (SW) algorithm [7], [8] is more practical for its capability to find similarities among subsequences, and it is a basis for many later algorithms. The two algorithms have similarities and dissimilarities [9]. To differentiate the two algorithms, some experts call NW algorithm as global SW algorithm, and SW algorithm as local SW algorithm. Though the two algorithms have dissimilarities, both of them suffer from the two mentioned problems: 1) time-consuming computation, and 2) dedicated hardware's inflexibility.

To attack the two problems for NW algorithm, C. W. Yu *et al.* presented an efficient architecture [10]. Later in 2005, P. Zhang *et al.* presented another high-performance accelerator card for NW algorithm [11]. Contrary to these work, we present a reconfigurable accelerator for the more practical SW algorithm. In our work, the first main contribution is that we introduce a temporary variable and modify SW algorithm into a new equivalent format which can improve its mapping efficiency to field-programmable gate array (FPGA) fabric. Another main contribution is that we utilize a special floor plan to cut down the routing delay of the critical path in the accelerator.

II. SW ALGORITHM

SW algorithm identifies similar subsequences using a dynamic programming (DP) method. In the DP method, SW algorithm compares two sequences by computing a distance representing the minimal cost to transfer one subsequence into another one. In the transformation, two edit operations, i.e., substitution and insertion/deletion (also called a gap operation), are possible. With a series of such operations, any sequence can be transformed into another sequence. The distance between two

sequences is defined as the smallest number of operations required to change one into the other.

Let us consider two sequences S of length l_1 and Q of length l_2 , SW algorithm computes similarity matrix $H(i, j)$ of two sequences ending at position S_i of S and position Q_j of Q respectively to identify similar subsequences. This basic recurrent computation is given by

$$\begin{aligned}
 H(i, j) &= \max \begin{cases} 0 \\ E(i, j) \\ F(i, j) \\ H(i-1, j-1) + Sbt(S_i, Q_j), \end{cases} \\
 &\quad \text{for } 1 \leq i \leq l_1, 1 \leq j \leq l_2 \\
 E(i, j) &= \max \begin{cases} H(i, j-1) - \alpha \\ E(i, j-1) - \beta, \end{cases} \\
 &\quad \text{for } 0 \leq i \leq l_1, 1 \leq j \leq l_2 \\
 F(i, j) &= \max \begin{cases} H(i-1, j) - \alpha \\ F(i-1, j) - \beta, \end{cases} \\
 &\quad \text{for } 1 \leq i \leq l_1, 0 \leq j \leq l_2
 \end{aligned} \quad (1)$$

where $Sbt(S_i, Q_j)$ is a substitution-cost table and initialization values are given by $H(i, 0) = E(i, 0) = H(0, j) = F(0, j) = 0$ for $0 \leq i \leq l_1, 0 \leq j \leq l_2$.

In the computation, an affine gap penalty $G(k)$ is adopted for k gaps' cost; i.e., $G(k) = \alpha + (k-1)\beta$, where $\beta \leq \alpha$, α is cost of the first gap, β is cost of each following gap. If $\alpha = \beta$, the recurrence can be simplified into a linear gap penalty.

After getting all similarity values in $H(i, j)$, we can reconstruct the optimal edit-transformation between the two sequences by a backtracking procedure from these values.

III. HARDWARE IMPLEMENTATION

A. Previous Work

Dedicated architectures developed for finding similarities among DNA sequences or protein sequences can be divided into two classes according to their hardware technologies.

- 1) **Application specific integrated circuit (ASIC)-based one:** Such hardware [3], [4] can provide the fastest means to run a particular algorithm with a very high processing element (PE) density without flexibility and with high design and producing cost. For these disadvantages, this kind of technology fades slowly out in bioinformatics' applications.
- 2) **FPGA-based one:** Such architecture [10], [12]–[17] is reconfigurable, generally slower and has lower PE density, but it is cheaper, more flexible, and can be regularly updated to state-of-art technology roughly according to Moor's law. These advantages make FPGA a very attractive alternative technology other than ASIC in bioinformatics' applications.

Most researchers mapped SW algorithm directly to a linear PE array [14], [15]. Such a PE (we call it a traditional PE) is shown in Fig. 1, where look-up table ("LUT") is used to store a substitution-cost table $Sbt(S_i, Q_j)$, "sign extend" component is used to make easy additions of two addends.

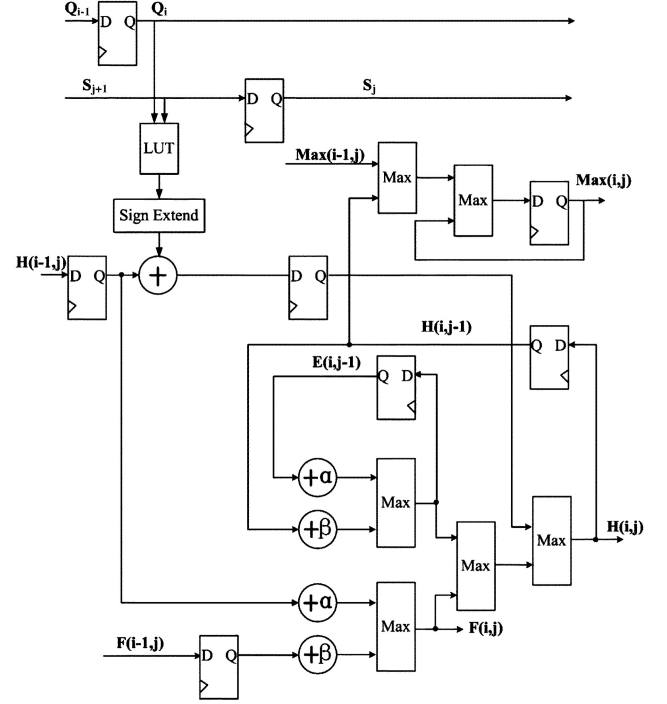


Fig. 1. Directly mapped PE.

In the PE array, each character Q_i of a query sequence is assigned to a PE, and a subject sequence S flows through the PE array. The query sequence keeps fixed until all subject sequences flow through the PE array.

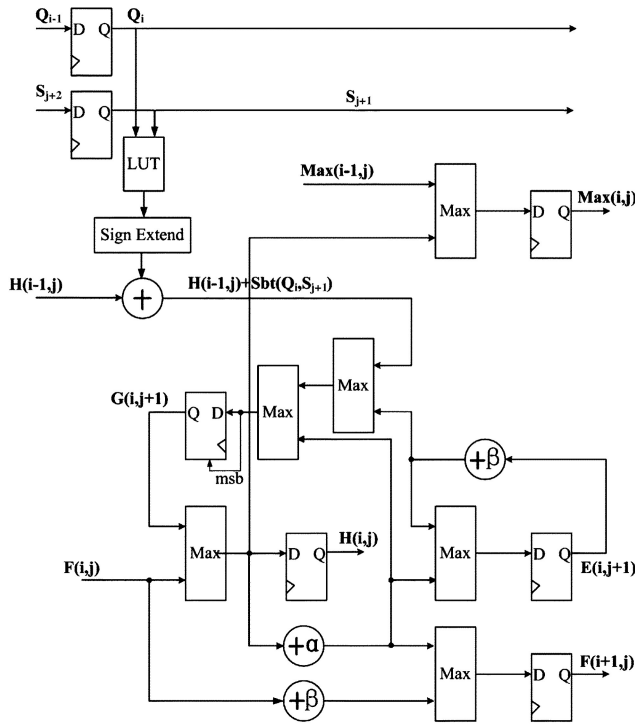
Obviously, the traditional PE has a critical path with three max operations and one addition operation, but its shortest path consists only of one max operation. Such unbalanced paths get low mapping efficiency of a PE to FPGA fabric.

B. Our Improved PE Implementation

We take advantage of FPGA fabric by introducing a new interim variable $G(i, j)$ in SW algorithm. With this modification, we can rewrite (1) as

$$\begin{aligned}
 H(i, j) &= \max \begin{cases} F(i, j) \\ G(i, j), \end{cases} \\
 &\quad \text{for } 1 \leq i \leq l_1, 1 \leq j \leq l_2 \\
 G(i, j) &= \max \begin{cases} 0 \\ E(i, j) \\ H(i-1, j-1) + Sbt(S_i, Q_j), \end{cases} \\
 &\quad \text{for } 1 \leq i \leq l_1, 1 \leq j \leq l_2 \\
 E(i, j) &= \max \begin{cases} H(i, j-1) - \alpha \\ E(i, j-1) - \beta, \end{cases} \\
 &\quad \text{for } 0 \leq i \leq l_1, 1 \leq j \leq l_2 \\
 F(i, j) &= \max \begin{cases} H(i-1, j) - \alpha \\ F(i-1, j) - \beta, \end{cases} \\
 &\quad \text{for } 1 \leq i \leq l_1, 0 \leq j \leq l_2.
 \end{aligned} \quad (2)$$

To adapt to the modification, the i th PE gets a mission during the j th clock cycle to compute the variables $H(i, j)$, $E(i, j+1)$, $G(i, j+1)$, and $F(i+1, j)$, where $H(i, j)$ and $F(i+1, j)$ are output to the $(i+1)$ th PE; $E(i, j+1)$ and $G(i, j+1)$ are used



in the $(j + 1)$ th clock cycle. The computation of $E(i, j + 1)$, $G(i, j + 1)$, and $F(i + 1, j)$ is

$$\begin{aligned} E(i, j+1) &= \max \begin{cases} H(i, j) - \alpha \\ E(i, j) - \beta \end{cases} \\ G(i, j+1) &= \max \begin{cases} 0 \\ H(i, j) - \alpha \\ E(i, j) - \beta \\ H(i-1, j) + Sbt(S_i, Q_{j+1}) \end{cases} \\ F(i+1, j) &= \max \begin{cases} H(i, j) - \alpha \\ F(i, j) - \beta \end{cases} \end{aligned} \quad (3)$$

Based on (3), the i th PE computes the variables $G(i, j)$ and $F(i, j)$ in advance, which is not like in the traditional (1).

$$\text{Max}(i, j) = \max \begin{cases} \text{Max}(i-1, j) \\ H(i, j) \end{cases} \quad (4)$$

A PE based on our improvements is shown in Fig. 2, where components “LUT” and “sign extend” have the same functions as those in Fig. 1; component most significant bit (“msb”) is used to reset $G(i, j + 1)$ to zero when most significant bit of $G(i, j)$ at previous cycle is equal to 1. This reset operation is corresponding to the “0” entry in the recurrent equation.

The diagram illustrates the architecture of a reconfigurable accelerator card. At the top, a large rectangular block represents the external memory, divided into several vertical slots, with the label "DDR SDRAM" centered. Below this, a double-headed arrow labeled "72" indicates a 72-bit data path to a central block labeled "DDR SDRAM interface". This central block is divided into three vertical sections: "JTAG I/F" on the left, "Altera traxtis FPGA" in the middle, and "ZBT SRAM interface" on the right. Within the FPGA section, there is a "Local Smith-Waterman algorithm PE array" which contains two "FIFO" blocks labeled "Subject" and "Query". Below these FIFOs is a "PCI-X controller". To the left of the central block, a "JTAG socket" is connected to the "JTAG I/F" section. Below the socket is an "EPC config FLASH" block, connected to the "EPC I/F" section of the central block via a double-headed arrow labeled "8". To the right of the central block, a "Reconfigurable accelerator card" label points to a stack of four rectangles labeled "36bit ZBT SRAM", which is connected to the "ZBT SRAM interface" section via a double-headed arrow labeled "144". At the bottom, a "PCI-X slot" is connected to the "PCI-X controller" via a double-headed arrow labeled "72".

The diagram illustrates the architecture of the proposed SoC. It features a central 'Local Smith-Waterman algorithm PE array' connected to four IO banks (IO bank 1, IO bank 2, IO bank 5, IO bank 6) via a 'DDR SDRAM interface' and a 'SRAM interface'. A 'PCI-X controller' is also connected to the IO banks. The architecture is divided into two main sections: 'IO bank 7' on the left and 'IO bank 8' on the right.

maximum calculation operators and one adder, which makes the critical PE path delay decrease about 1/4.

Our accelerator adopts a 0.13- μm FPGA device Altera EP1S30 with 30 000 logic elements (LEs) [18], it is hosted by a PCI-X bus. Its architecture is shown in Fig. 3.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 24, 2025 at 12:49:25 UTC from IEEE Xplore. Restrictions apply.

To evaluate our accelerator performance, we should first get the accelerator's calculation time T_{Acc} by

$$T_{Acc} = T_{PE} \times N + T_{Rout} \times (N - 1) + T_{In} + T_{Out} \quad (5)$$

where T_{PE} is time for a data flows through a PE; T_{Rout} is the average routing time between two neighbor PEs; T_{In} is time for a data flows in the first PE by PCI-X interface; T_{Out} is time for a data flows out the last PE by PCI-X interface; N is the number of PEs in the FPGA.

For our improved PE, T_{PE} decreases about 1/4 as mentioned above. With the special floor plan different from that in [15], T_{Rout} , T_{In} , and T_{Out} can reach the smallest compared to other implementations. At the same time, our improved PE uses the same amount of logic resource as the traditional one, so PE number N keeps unchanged in the same device if using the same optimization techniques. As a result, this special floor plan can further decrease T_{Acc} time. At last, compared to a traditional implementation using neither the modified equation nor our zigzag floor plan, T_{Acc} can totally decrease about 1/3.

In our accelerator, PCI-X controller is used for communication between a host computer and the accelerator. Query and subject sequence data are transferred from the main memory by direct memory access (DMA). Two FIFOs of on-chip RAM buffer the input data, and convert 64-bit data to 2-bit data representing each nucleotide. These two FIFOs are asynchronous ones to match the PCI-X controller and PE array operating in different clock domains.

Thanking to reconfigurability, the 20-bit word width for the maximum similarity value according to concrete applications is customizable. It ranges from -2^{19} to $(2^{19} - 1)$, which is sufficient for most bio-sequence search applications.

Sequence length should be dealt with carefully. There is no limitation on the length of a subject sequence for the sequence can flow into the accelerator freely as needed. However, each time the accelerator can only deal with one query sequence with length equal to or less than the PE number N . If the length is above N , we trickly partition the sequence into segments with length equal to or less than N . Then similarity matrix of each segment is calculated based on interim results stored in the on-board 36 Mbit SRAM or 256 MB DDR SDRAM. The just gotten similarity matrix is interim results for a neighbor segment of the just calculated segment. When a query sequence is too long for ZBT SRAM to accommodate the interim results, one can switch to use DDR SDRAM.

V. PERFORMANCE EVALUATION

To measure the accelerator's performance, we adopt the maximum attainable number of dynamic programming cell updates per second (CUPS) which is a rough performance measure in computational biology [19]. A cell update includes all operations on one entry of matrix H ; i.e., all comparisons, additions, and maximum computations. Our reconfigurable accelerator is up to 6.6 GCUPS, which is gotten by multiplying the PE number N by its running frequency.

A. Comparison to a Desktop Platform

To get software system time T_{PC} , we run a standard SW program *SSearch* on a desktop PC with a 2.8-GHz Xeon processor and 4-GB memory. To get whole system time T_{Sys_acc} with our accelerator, we plug the accelerator on the desktop PC and use

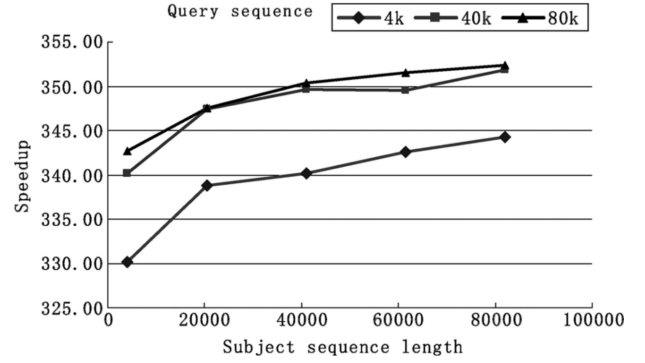


Fig. 5. Performance comparison between our accelerator and a desktop PC.

TABLE I
PERFORMANCE COMPARISON TO A TRADITIONAL IMPLEMENTATION

	Our Imp.	Transferred Traditional Imp.
Word width (bit)	20	20
LEs needed by PE	~320	~320
Maximum PEs	80	80
Maximum frequency	~82MHz	~55MHz
CUPS performance	~6.6G	~4.4G

the accelerator to execute SW algorithm. Actual system time T_{Sys_acc} includes not only the calculation time T_{Acc} by PE array (including communication time), but also pre-processing time T_{Pre} , post-processing expense T_{Post} , and PE array initialization time T_{Init} . Fortunately, T_{PC} includes the same pre-processing and post-processing expense as T_{Sys_acc} under our test case. The initialization time T_{Init} is less than $0.038 \mu s (\approx ((80 \times 2) / (66 \times 64)) \mu s)$ for the high bandwidth PCI-X bus, thus it is neglectable compared to T_{Acc} . Upon these conditions, speedup of our accelerator to the desktop PC can be computed by T_{PC} / T_{Sys_acc} .

In our test case, sequences are randomly chosen from GeneBank. Their length ranges from 4 to 80 kbp. The experimental results are shown in Fig. 5.

From Fig. 5, we can conclude that our accelerator has more than 330 speedup as compared to the desktop platform. Longer is a sequence, speedup is higher. Speedup increases slowly after a query sequence is above a certain length (e.g., 40 kbp). The results correctly reflect the facts that T_{Init} is neglectable and T_{Acc} dominates in T_{Sys_acc} . The dominance decreases slowly along with sequence length increases, and keeps almost stable when the length exceeds a certain length. The high speedup greatly outperforms the speed improvement predicted by Moor's law for a general processor, not to say we can update the FPGA device roughly obeying to Moor's law.

B. Comparison to a Transferred Traditional Implementation

Traditional SW algorithm implementation [14], [15] used an XC2V6000 device. To make performance comparison between it and ours, we transferred the traditional implementation to the same FPGA as used by our accelerator. The transfer work is done with the same optimization techniques (i.e., using the same interface, etc.) except not utilizing our improved SW algorithm and the special floor plan. After transformation, its performance comparison with ours is shown in Table I.

The comparison demonstrates that the maximum chip frequency in the same Altera device is improved due to the modified equation and the special floor plan. Besides, the peak performance of our accelerator improved 50% to that of the transferred traditional implementation.

C. Comparison to the Most Recent Advance

Recently, Court and Herbordt provided a family of FPGA-based accelerators for NW algorithm and SW algorithm [9]. Though they claimed that their accelerator family got very high performance, they focused on the flexibility to generate a large family of accelerators for DP algorithm by multiple, interchangeable implementations for three components. The three components addressed different parts of the approximate string matching problem without using our two techniques.

Quantitatively, we compare our implementation with their implementations by clock speed. Our implementation is for SW algorithm and gets about a clock speed of 12.2 ns. The speed is about 9% faster than the 13.3-ns clock speed of T. V. Court's fastest implementation for SW algorithm. Moreover, our implementation is faster than the 12.9-ns clock speed of T. V. Court's fastest implementation for NW algorithm. Upon these, our implementation using both a modified equation and a special floor plan is better than the most recent advance for both NW and SW algorithm acceleration.

VI. DISCUSSION

Some design skills are employed to make our accelerator a common platform for both NW [11] and SW algorithm.

A. Reconfigurability

For easy switch between NW algorithm and SW algorithm, reconfigurability is necessary. In addition, as to SW algorithm acceleration, when query sequence or bio-sequence database change, the internal data width and other parameters of our accelerator need to be reconfigured. For such reconfiguration, we adopt an overall configuration by the fast passive parallel (FPP) configuration scheme to reduce configuration time. Furthermore, we save more time by storing important registers' contents in on-board SRAM or DDR SDRAM other than getting them by reconfiguration. With these techniques, the configuration process takes approximately 20 ms.

B. Using LogicLock Methodology

Logic function of most commercial FPGA devices using CMOS SRAM technology does not change until configuration data corresponding to one area is updated. To alleviate this disadvantage for an implementation with multiple modules, we apply Altera LogicLock methodology [20]. With LogicLock technique, a logic module needs only to be optimized once, and back-annotation of the LogicLock region can preserve the optimized performance and keep mapped area unchanged. In addition, configuration data can be reduced by the method. The gain is from that only configuration data of the changed modules needs to be transferred in a new configuration process. LogicLock regions in our accelerator include all interface parts for different SW algorithm implementation/applications.

C. Dynamically Updating Substitution Table

Different bioinformatics scientists often adopt different substitution-cost table $Sbt(S_i, Q_j)$, which requires a frequently tedious configuration process. To solve this, we store substitution-cost table in some FPGA embedded RAM blocks. Users can access and update the table using Quartus II software dynami-

cally by utilizing Altera in-system updating memory's characteristics.

VII. CONCLUSION AND FUTURE WORK

We presented a reconfigurable accelerator for SW algorithm based on a modification to original SW algorithm and using a special zigzag floor plan. The two special techniques make the contribution to improve performance of accelerating SW algorithm. Experiments demonstrates that our accelerator provides interesting gains than several efficient software or hardware implementations.

Our two sharable optimization techniques can be easily applied to hardware acceleration for other computing intensive algorithms, for example, a multiple sequence alignment algorithm. Now we are on the way to extend our accelerator for such an algorithm and other bioinformatics' applications.

ACKNOWLEDGMENT

The authors would like to thank the editors and the anonymous reviewers for their helpful comments and suggestions.

REFERENCES

- [1] S. Altschul, W. Gish, W. Miller, E. Myers, and D. Lipman, "Basic local alignment search tool," *J. Molec. Biol.*, vol. 215, pp. 403–410, 1990.
- [2] W. Pearson, "Comparison of methods for searching protein sequence databases," *Protein Sci.*, vol. 4, no. 6, pp. 1145–1160, Jun. 1995.
- [3] R. K. Singh, D. L. Hoffman, S. G. Tell, and C. T. White, "BioSCAN: A network sharable computational resource for searching biosequence databases," *Bioinformatics*, vol. 12, no. 3, pp. 191–196, Jun. 1996.
- [4] P. Guerdoux-Jamet and D. Lavenier, "SAMBAs: Hardware accelerator for biological sequence comparison," *Bioinf.*, vol. 13, pp. 609–615, 1997.
- [5] [Online]. Available: <http://www.timelogic.com>
- [6] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acids sequences of two proteins," *J. Molec. Biol.*, vol. 48, pp. 443–453, 1970.
- [7] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequences," *J. Molec. Biol.*, vol. 147, no. 1, pp. 195–197, 1981.
- [8] O. Gotoh, "An improved algorithm for matching biological sequences," *J. Molec. Biol.*, vol. 162, no. 3, pp. 705–708, Dec. 1982.
- [9] T. V. Court and M. C. Herbordt, "Families of FPGA-based accelerators for approximate string matching," *Microprocess. Microsyst.*, vol. 31, no. 2, pp. 135–145, 2007.
- [10] C. W. Yu, K. H. Kwong, K. H. Lee, and P. H. W. Leong, "A Smith-Waterman systolic cell," in *Proc. 13th Int. Workshop Field Programmable Logic Appl. (FPL'03)*, 2003, pp. 375–384.
- [11] P. Zhang, X. Liu, and X. Jiang, "An implementation of reconfigurable computing accelerator card oriented bioinformatics," *J. Comput. Res. Dev.*, vol. 42, no. 6, pp. 930–937, 2005.
- [12] D. T. Hoang, "Searching genetic databases on splash 2," in *Proc. 1st IEEE Workshop on Field-Programmable Custom Computing Machines*, Apr. 1996, pp. 185–192.
- [13] Y. Yamaguchi, Y. Miyajima, T. Maruyama, and A. Konagaya, "High speed homology search using run-time reconfiguration," in *Proc. 12th Int. Workshop Field Programmable Logic Appl. (FPL'02)*, 2002, pp. 281–291.
- [14] T. F. Oliver, B. Schmidt, and D. L. Maskell, "Hyper customized processors for bio-sequence database scanning on FPGAs," in *Proc. 13th Int. Symp. Field-Programmable Gate Arrays*, Feb. 2005, pp. 229–237.
- [15] T. Oliver, B. Schmidt, and D. L. Maskell, "Reconfigurable architectures for bio-sequence database scanning on FPGAs," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 52, no. 12, pp. 851–855, Dec. 2005.
- [16] S. Masuno, T. Maruyama, Y. Yamaguchi, and A. Konagaya, "Multidimensional dynamic programming for homology search," in *Proc. Int. Workshop Field Programmable Logic Appl. (FPL'05)*, 2005, pp. 173–178.
- [17] S. Masuno, T. Maruyama, Y. Yamaguchi, and A. Konagaya, "Multidimensional dynamic programming for homology search on distributed systems," in *Proc. Euro. Conf. Paral. Comp.*, 2006, pp. 1127–1137.
- [18] Stratix Device Handbook, Altera Inc., Santa Clara, CA, Apr. 2003.
- [19] D. Lavenier, "Dedicated hardware for biological sequence comparison," *J. Universal Comput. Sci.*, vol. 2, no. 2, pp. 77–86, Feb. 1996.
- [20] Quartus II Handbook Altera Inc., Santa Clara, CA, Feb. 2004.